Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

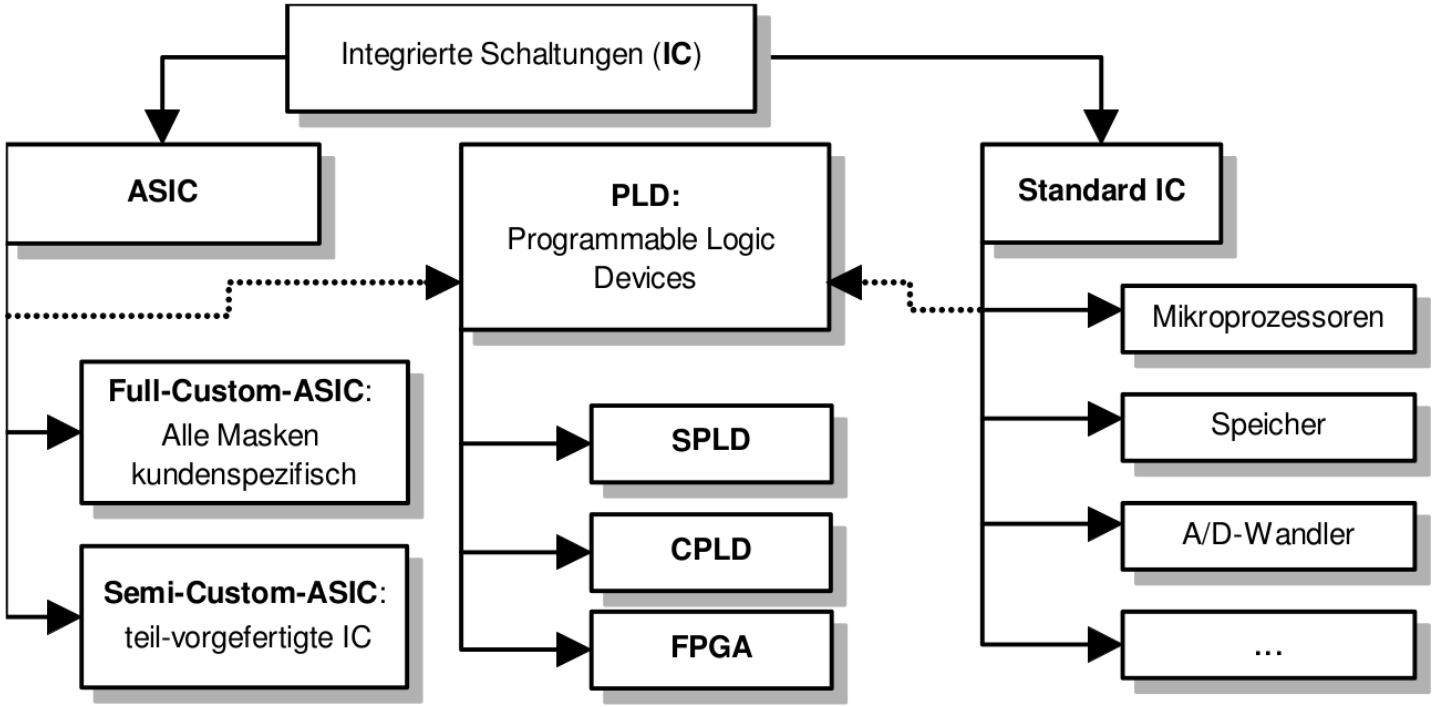
# Hardware-Software-Systeme

## Kapitel 07 - ASIC-Design

M.Sc. Moritz Weißbrich, Prof. Dr.-Ing. Guillermo Payá Vayá, 27.01.2026

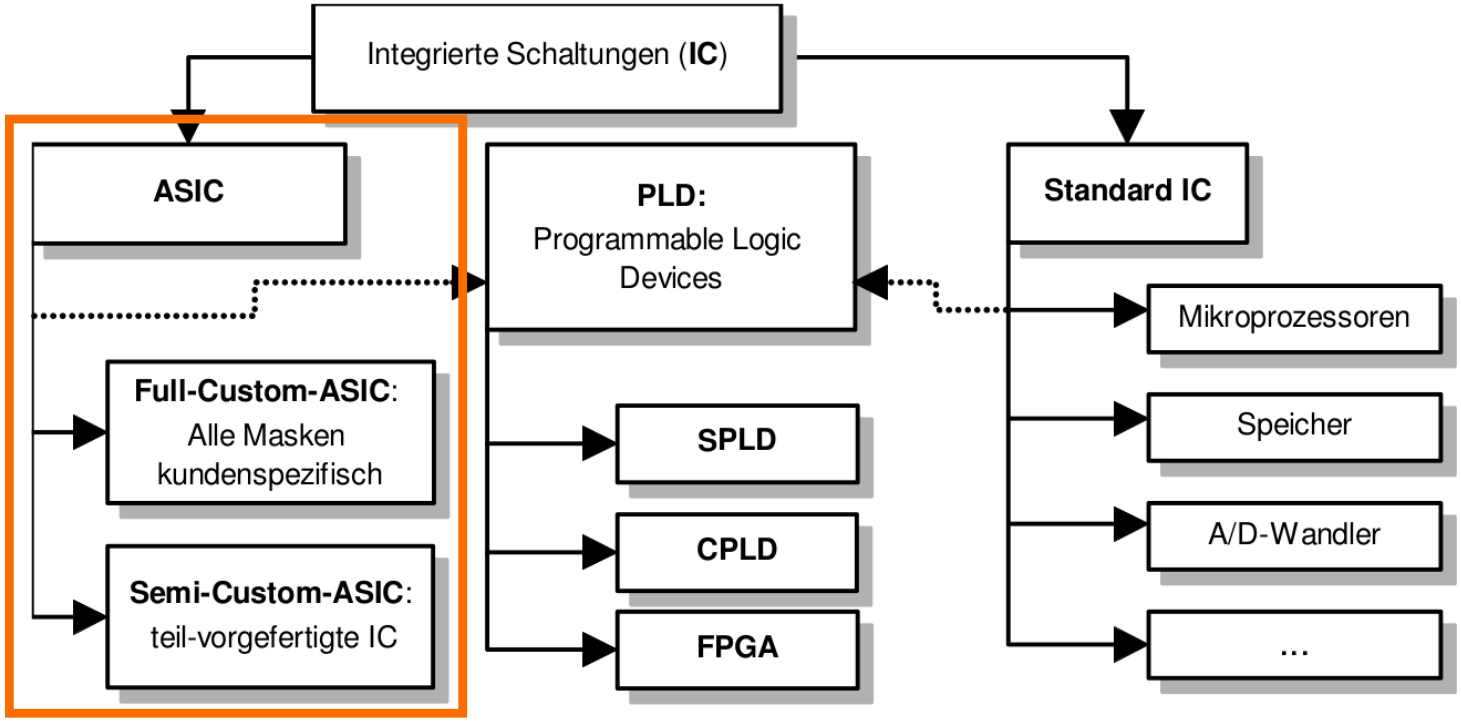
# Application-Specific IC (ASIC)

- ASICs werden für spezifische Anwendungsfälle entwickelt.
- Sie erzielen für diese bessere Hardwareeigenschaften (Performance, Fläche, Verlustleistung, etc.) als Standardkomponenten oder PLDs.



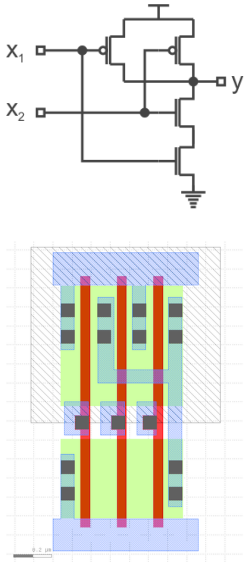
# Application-Specific IC (ASIC)

- ASICs werden für spezifische Anwendungsfälle entwickelt.
- Sie erzielen für diese bessere Hardwareeigenschaften (Performance, Fläche, Verlustleistung, etc.) als Standardkomponenten oder PLDs.



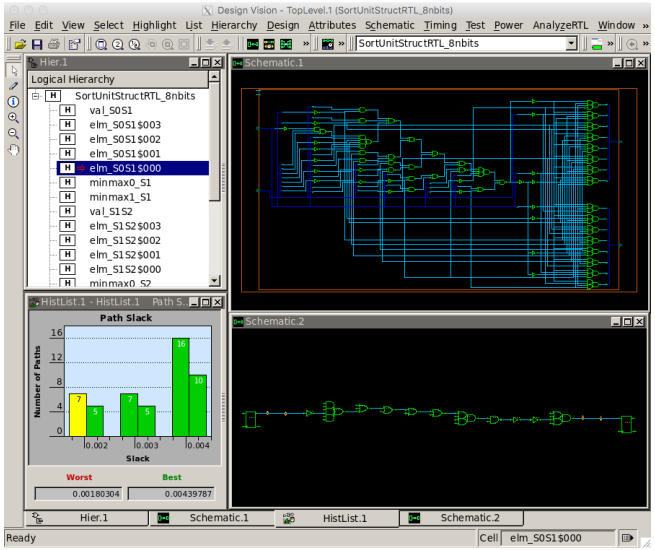
# Application-Specific IC (ASIC)

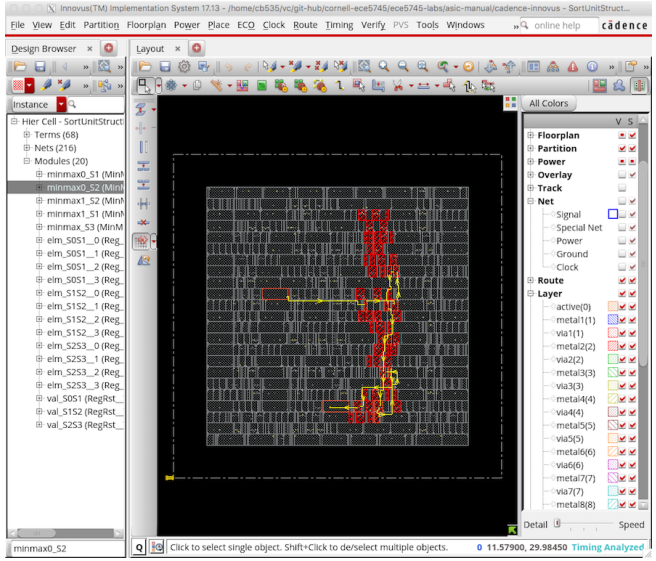
*Full-Custom ASIC*




Entwurf auf Transistorebene

*Semi-Custom ASIC*




Vorbereitete *Standardzell-Bibliotheken*, ASIC-spezifischer Entwurf beschränkt sich im Wesentlichen auf *Synthese*, *Platzierung* und *Verdrahtung*

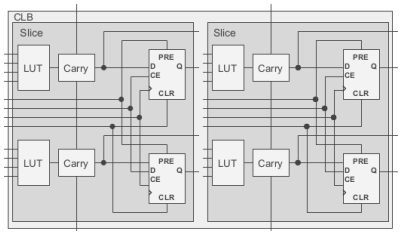
# ASICs vs. FPGAs

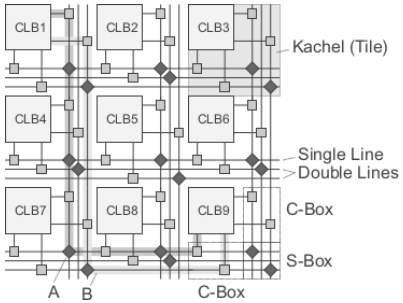
*Standardzell-ASIC*

- Nur benötigte Logik wird auch physisch implementiert
- Routing wird spezifisch erstellt
- Mit Ausnahme von Nutzung vorgegebener Standardzellbibliotheken und Reihenanordnung prinzipiell *keine* Strukturvorgaben

→ *"Anwendungsspezifisches Design"*

*Field Programmable Gate Arrays (FPGA)*



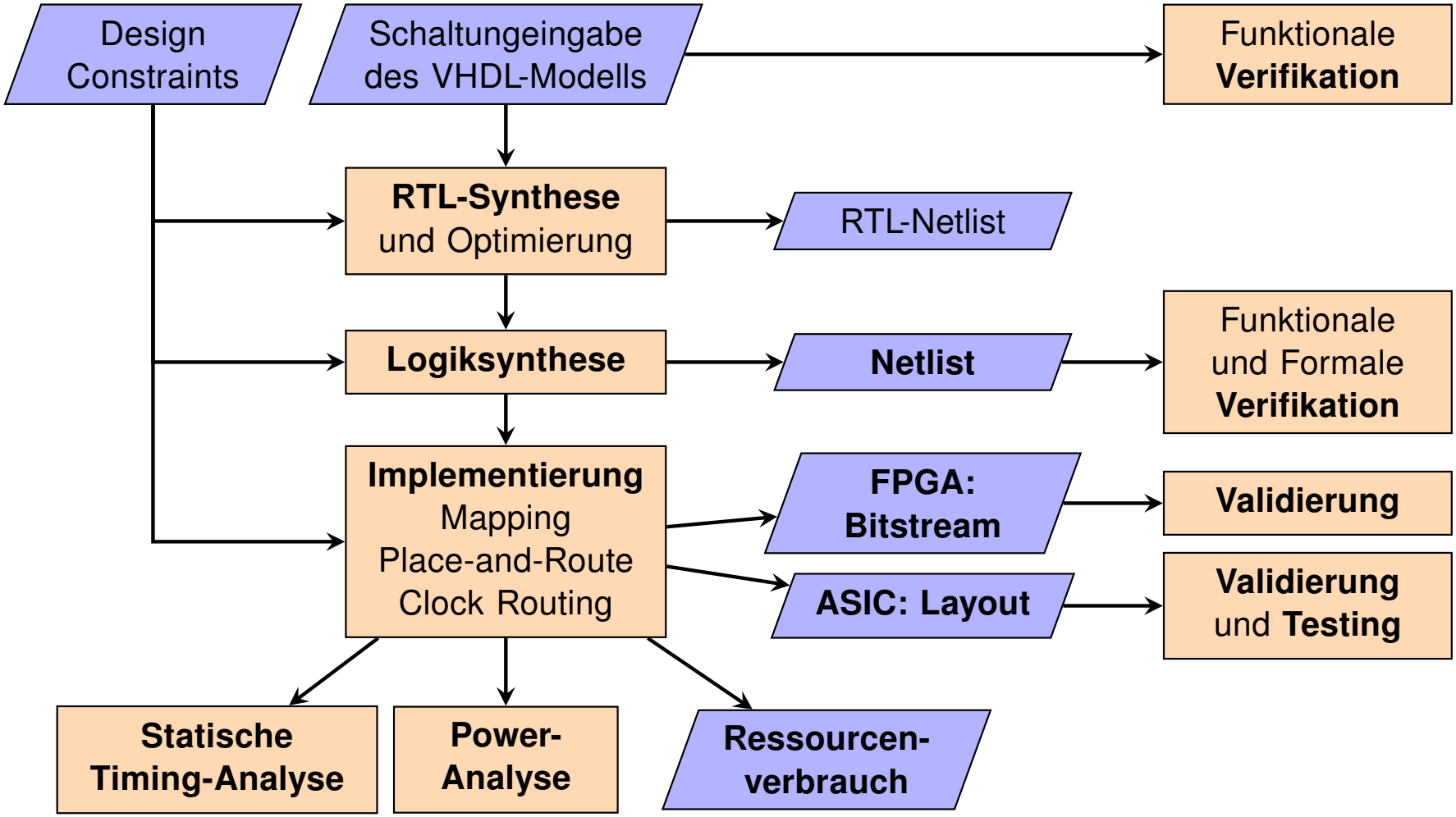

- Logikelemente strukturell vorgegeben (CLBs, Register, LUTs)
- Routing strukturell vorgegeben (Routing Switches, C-Boxes)
- Funktion durch Konfiguration

→ *"Konfigurierbare Architektur"*

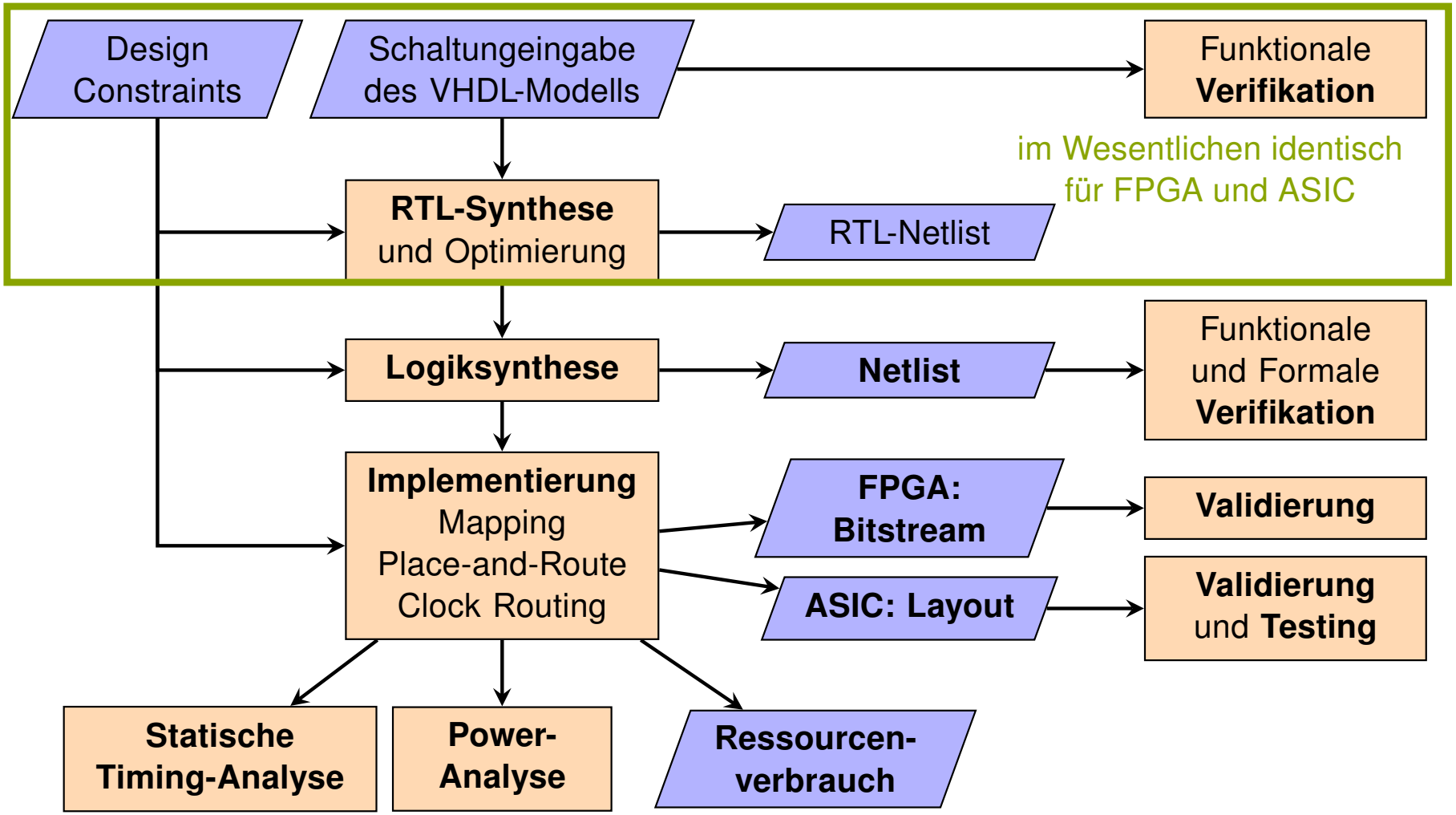
# ASICs vs. FPGAs

| *Eigenschaft* | ASIC | FPGA |
|---|---|---|
| Entwurfsflexibilität | + | - |
| Geschwindigkeit der Schaltung | + | - |
| Siliziumfläche der Schaltung | + | - |
| Verlustleistung der Schaltung | + | - |
| Entwurfsaufwand | - | + |
| Rekonfigurierbarkeit | - | + |
| Time-to-Market | - | + |

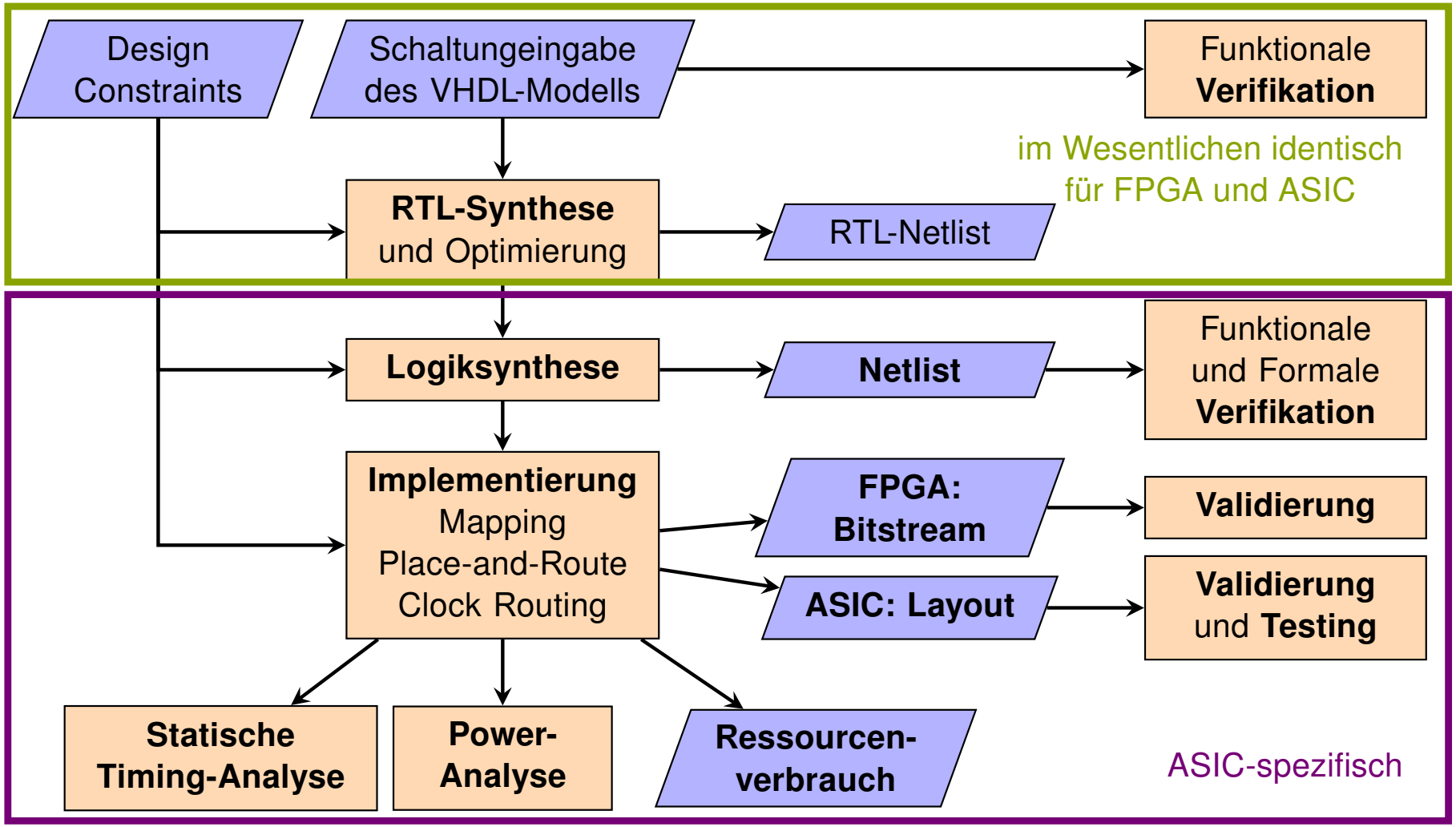
# Design Flow


Design Constraints
Schaltungeingabe des VHDL-Modells
Funktionale Verifikation
RTL-Synthese und Optimierung
RTL-Netlist
Logiksynthese
Netlist
Funktionale und Formale Verifikation
Implementierung
Mapping
Place-and-Route
Clock Routing
FPGA: Bitstream
Validierung
ASIC: Layout
Validierung und Testing
Statische Timing-Analyse
Power-Analyse
Ressourcen-verbrauch

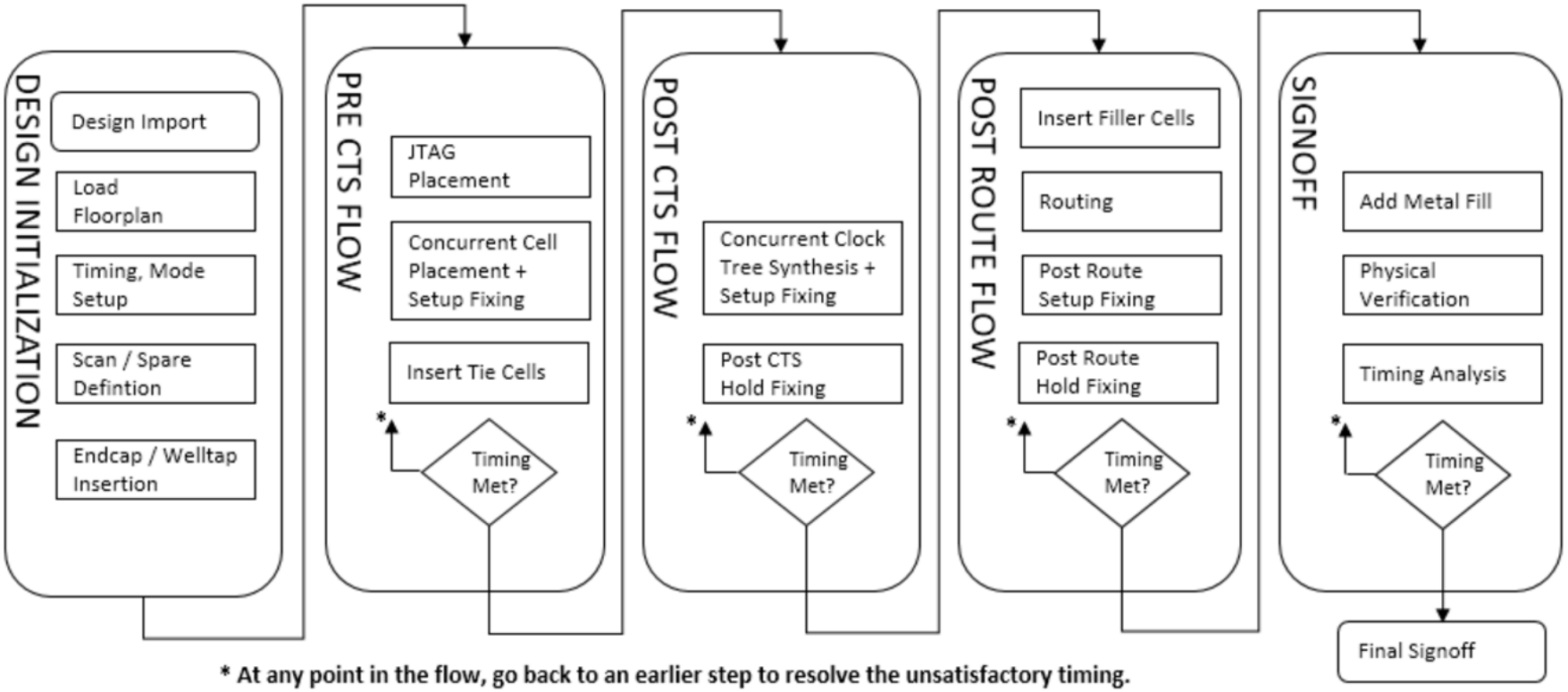
# Design Flow


Design Constraints
Schaltungeingabe des VHDL-Modells
Funktionale Verifikation
im Wesentlichen identisch für FPGA und ASIC
RTL-Synthese und Optimierung
RTL-Netlist
Logiksynthese
Netlist
Funktionale und Formale Verifikation
Implementierung
Mapping
Place-and-Route
Clock Routing
FPGA: Bitstream
Validierung
ASIC: Layout
Validierung und Testing
Statische Timing-Analyse
Power-Analyse
Ressourcen-verbrauch

# Design Flow


Design Constraints
Schaltungeingabe des VHDL-Modells
Funktionale Verifikation
im Wesentlichen identisch für FPGA und ASIC
RTL-Synthese und Optimierung
RTL-Netlist
Logiksynthese
Netlist
Funktionale und Formale Verifikation
Implementierung Mapping Place-and-Route Clock Routing
FPGA: Bitstream
Validierung
ASIC: Layout
Validierung und Testing
Statische Timing-Analyse
Power-Analyse
Ressourcen-verbrauch
ASIC-spezifisch

# ASIC-spezifischer Backend-Flow (I)




* At any point in the flow, go back to an earlier step to resolve the unsatisfactory timing.

# ASIC-spezifischer Backend-Flow (II)

- Im **Floorplanning** werden Position, Größe und Form von funktionalen Modulen, sowie die Platzierung von *Makrozellen* durch den Designer sinnvoll vorgeplant. Dies dient der Unterstützung der Implementierungstools und der Verbesserung der Implementierungsergebnisse. Floorplanning enthält ebenfalls die Planung der Reihenanordnung für Standardzellen sowie das *Power Planning*. Meist sind mehrere Iterationen für den Floorplan erforderlich, bis zufriedenstellende Ergebnisse bei der Zellplatzierung und für das Timing erreicht werden.
- Die **Platzierung (Placement)** von Standardzellen berücksichtigt den gewählten Floorplan sowie die Hierarchie und Verbindungsstruktur von Signalen zwischen den Zellen. Auch wird darauf geachtet, dass keine Zellüberlappungen entstehen und Abstandsregeln für die spätere Verdrahtung eingehalten werden.

# ASIC-spezifischer Backend-Flow (III)

- **Clock Tree Synthesis (CTS)** erzeugt und platziert notwendige Buffer-Logik, um Taktsignale von der Taktquelle an alle taktgesteuerten Zellen des gesamten Chips zu verteilen. Hierbei ist es notwendig, alle Pfade des Clock Trees so zu balancieren, dass der *Taktversatz* (clock skew) minimiert wird. So werden Funktionsfehler der Logik durch Timingprobleme (Setup- und Hold-Verletzungen) vermieden.
- Der Schritt der **Verdrahtung (Routing)** erzeugt die Signalverbindungen (Wires) zwischen Ein- und Ausgängen der Zellen. Ziel des Routings ist es, alle notwendigen Verbindungen ohne Kurzschlüsse oder Verletzung der Abstandsregeln herzustellen.
- Zusätzliche ASIC-spezifische Schritte können eingefügt werden, wie z.B. das **Metal Filling**, welches die Metalldichte in den Verdrahtungsebenen homogener gestaltet. Dies erhöht die Ausbeute bzw. reduziert vermeidbare Fertigungsfehler.

# Statische Timing-Analyse (STA)

Das Prinzip der STA ist für FPGA- und ASIC-Implementierungen identisch. Jedoch ist folgendes zu beachten:

- **FPGA:** Da die konfigurierbare Routing-Infrastruktur fest vorgegeben ist und viele Routing Switches und Connection Boxes als verzögernde Elemente im Signalweg liegen, ist es nicht selten, dass *bis zu 80% der Pfadverzögerung* durch das Routing entstehen!
- **ASIC:** Im Gegensatz zum FPGA erlaubt die spezifisch erstellte Verdrahtung in ASICs direkte (Metall-)Verbindungen zwischen Standardzellen, sodass ein wesentlich größerer Spielraum für die Optimierung der Verdrahtung besteht. Das Verhältnis zwischen Gatterverzögerung der Logik und Verzögerung durch das Routing fällt damit ausgewogener aus. ASIC-Implementierungen erreichen damit *wesentlich höhere Taktfrequenzen* als äquivalente FPGA-Implementierungen einer logischen Schaltung.

# Literatur


- Holger Blume, "Vorlesungsskript: FPGA-Entwurfstechnik", Leibniz Universität Hannover, 2020
- Frank Kesel, "FPGA Hardware-Entwurf. Schaltungs- und System-Design mit VHDL und C/C++", Walter de Gruyter GmbH, 2018
- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Cadence Design Systems, Inc., "Innovus User Guide 21.12", 2021.